CHAPTER 04

Microservices and event-driven architectures

For Business and Technical Analysts in international commercial banking

Antonio Piemontese

October 2026



CHAPTER 04

Learning objectives

01 **Evaluate service boundaries**

Connect business capabilities, data ownership and release needs.

02 **Distinguish interaction choices**

Explain commands, events, queries and asynchronous handoffs.

03 **Trace delivery and recovery**

Reason about duplicates, ordering and incomplete business effects.

04 **Specify contracts and evidence**

Name operational owners and useful tests for a distributed workflow.

Microservices and event-driven architecture

Microservices

Group behaviour into cohesive business services.

Support compatible changes through separate releases.

Keep service data behind owned contracts.

Event-driven architecture

Producers report facts that have occurred.

Consumers react according to their responsibilities.

Asynchronous handoffs allow later completion.

Modular apps can publish events. Microservices can use request-response.

Service boundaries and data ownership

A useful service boundary groups behaviour, business meaning and the state it must protect.

Application intake and notification delivery can have different release and operating needs.

Business capability

Keep related rules and behaviour together.

Domain model

A bounded context defines where a model applies.

Data and consistency

Keep private state behind explicit contracts.

Accountable ownership

Name who changes, operates and recovers the service.

HYPOTHETICAL SCENARIO EXTENSION

Banking pilot: selective service extraction

New release need

A notification team now needs independent releases. It would own message formatting and delivery records for the one-country pilot.

Continuing boundaries

Intake and workflow stay together.

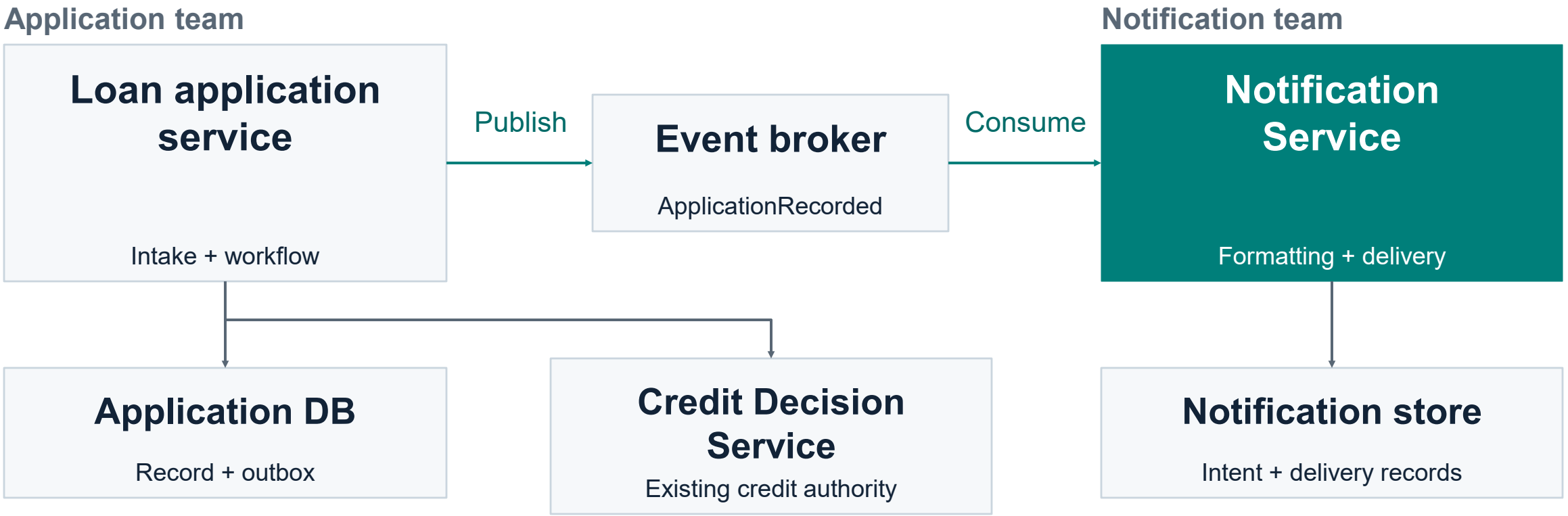
Existing credit authority remains.

Notification state stays separate.

Decision to evaluate	Does release independence justify another service and its operating work?

HYPOTHETICAL SERVICE DESIGN

Proposed service interactions



Each service owns its data. The application retains the existing credit contract. Identity and customer integrations are omitted.

MESSAGE MEANING

Commands, events and queries

Intent	Example	Business meaning
Command	SendReceiptNotification	Requests an action from a responsible receiver
Event	ApplicationRecorded	Reports that an application was recorded
Query	GetApplicationStatus	Requests information about current state

Commands can travel asynchronously. Event names describe facts that have occurred.

Synchronous and asynchronous interactions

Waiting for a result

The caller sets a deadline for a usable result.

Dependency failures affect this interaction.

Long call chains add coordination and latency.

Durable asynchronous handoff

A durable handoff records work for later processing.

Consumers complete work later, within limits.

Progress and expiry need explicit contracts.

Choose the interaction style for each business boundary. Both styles can coexist.

Queues, publish–subscribe and streams

Pattern	Consumption model	Typical use
Work queue	Workers compete for each delivery	Distribute notification work
Publish–subscribe	Each matching subscription receives its copy	Notify several capabilities
Retained stream	Independent readers track progress	Rebuild views from retained events

Patterns overlap. Retention, delivery and ordering depend on the broker and its configuration.

ILLUSTRATIVE EVENT CONTRACT

Event identity, meaning and evolution

Field	Illustrative value
specversion	1.0
id	E-884
source	/loan-applications
type	bank.loan.recorded.v1
subject	L-1042
data	{"applicationId":"L-1042"}

Identity and meaning

Use source + id for duplicates. The producer defines the fact.

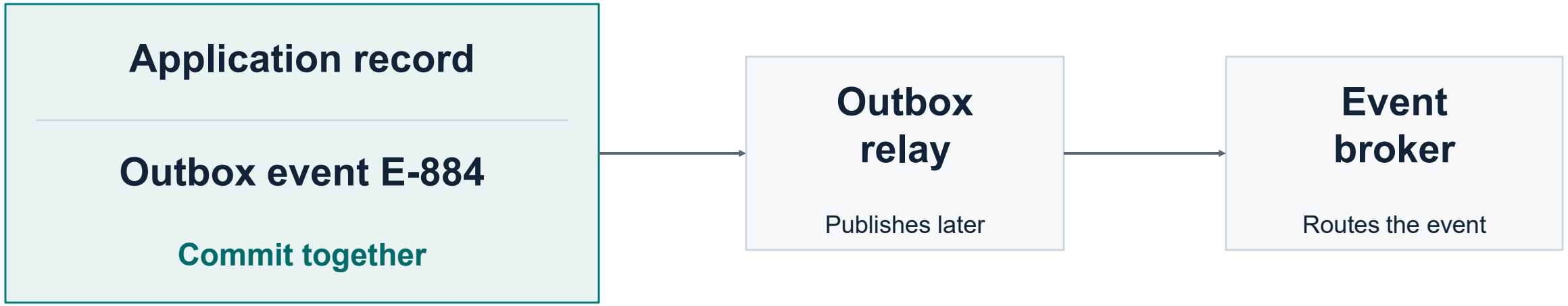
Schema evolution

specversion: envelope format. Business schemas evolve separately.

Share only the customer data that consumers are authorised to use.

Reliable publication with an outbox

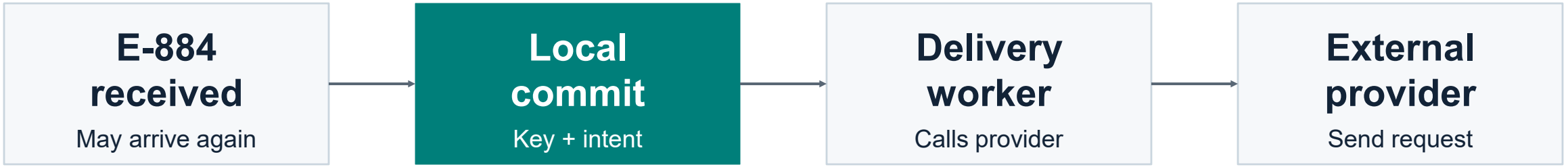
Application database



A lost publish acknowledgement can lead the relay to send the same event again.

Receipt follows the local commit. Publication can happen later and may repeat.

Duplicate delivery and external effects



Duplicate key: no new notification intent.
New key: commit key and intent together.

A provider timeout can leave delivery uncertain after the local commit.

Review question Which provider contract makes a retry safe?

Ordering and eventual consistency

Events may reach consumers later or out of order. Each state view needs explicit update and recovery rules.

Application and notification status have different owners and completion times.

Ordering scope

Specify the entity, partition and processing rule.

Version checks

Prevent stale updates and invalid transitions.

Gaps and recovery

Detect missing work and reconcile from an authority.

Freshness

Set acceptable delay and escalation rules.

Choreography, orchestration and sagas

Choreography

Participants react to events with local rules.

Independent reactions reduce direct coordination.

The full workflow is harder to reconstruct.

Orchestration

An explicit owner tracks workflow progress.

The coordinator issues commands to participants.

Its state and recovery become part of the design.

A saga coordinates local transactions. Compensation follows explicit business rules.

Retries, quarantine and replay

Failure or risk	Control	Evidence
Temporary failure	Bounded retries with backoff	Event completes or escalates
Unprocessable event	Quarantine, diagnose and repair	Named owner and safe redrive
Replay or redrive	Preserve event ID and check prior effects	No duplicate or stale updates

Recovery needs an owner, preserved event identity and evidence of completed business effects.

Operating a distributed workflow

A distributed workflow needs evidence of business progress and clear responsibility for recovery.

Follow the application, its event and the notification intent through each handoff.

Business outcomes

Track pending work and uncertain delivery.

Traceability

Carry application, event and correlation identity.

Security

Restrict publishers, consumers and replay access.

Contracts and owners

Test compatibility and name the recovery owner.

HYPOTHETICAL DECISION RECORD

Selective decomposition decision

ADR-004 Notification service boundary

Status: Proposed

Context

The scenario adds a notification team with its own release schedule. Intake and workflow remain together.

Decision

Extract a Notification Service with owned data and an event contract, subject to delivery and recovery evidence.

Consequences

Independent notification releases.

More operating work and later notification completion. Test provider retries and recovery.

GROUP EXERCISE

12 MINUTES

Exercise: duplicates and uncertain delivery

Notification handoff failure

E-884 for application L-1042 arrives twice.

The provider times out after a send request. Its outcome is unknown.

Your output

01 Define the identity and local commit.

02 Set the provider retry policy.

03 Specify replay and recovery evidence.

04 Separate customer and notification status.

Recorded application: Pending assessment. Notification delivery has a separate status.

EXERCISE DEBRIEF

Debrief: reliable business effects

01

One local intent

Commit a unique event key and the pending notification together.

02

Provider uncertainty

Use supported idempotency or reconciliation, with an explicit fallback.

03

Controlled replay

Preserve event identity and check effects already completed.

04

Truthful customer status

Keep durable receipt, credit assessment and notification status separate.

REFERENCE MATERIAL

References and further reading

Microservices and service boundaries

James Lewis, Martin Fowler and Microsoft Architecture Center

- Microservices Service boundaries

Event context and interface contracts

CloudEvents and AsyncAPI

- CloudEvents specification AsyncAPI specification

Reliable publication and consumption

Chris Richardson, Microservices.io

- Transactional outbox Idempotent consumer

Primary references. Topic-specific sources and teaching context appear in the speaker notes.